

Hermes + Ornith — Local AI Brain

Setup Runbook · Headless Windows PC (Quadro RTX 4000 + GTX 1060) · June 29, 2026

What you're building

The rackmount PC becomes a self-contained AI brain. Ollama serves the model(s) locally; Hermes runs on the same box, calls Ollama over its OpenAI-compatible API, and talks to you through Telegram. Your Mac and phone are just Telegram clients — the PC can run with no one logged in and still answer you from anywhere. Tailscale is only for getting into the box to manage it; the AI traffic never needs it.

```
Telegram (phone / Mac)
  | (Telegram servers, internet)
  v
+----- Windows PC (headless) -----+
| Hermes CLI --/v1--> Ollama #1 :11434 -> ornith:9b on RTX 4000 (8GB)|
|               --/v1--> Ollama #2 :11435 -> small+embed on GTX 1060 (6GB)|
+-----+
      ^
      | Tailscale + AnyDesk/RDP (admin only)
      | You, for maintenance
```

Note: Do the stages **in order** and stop at the end of each to confirm it works before moving on. Stages 1–3 get you a fully working Telegram brain on the RTX 4000. The GTX 1060 sidecar (Stage 4) and autostart (Stage 5) are add-ons — don't start them until 1–3 are solid.

Before you start

- You're working on the PC over AnyDesk. Run every PowerShell block in an **Administrator** PowerShell unless noted (right-click PowerShell → Run as administrator).
- Have your Telegram app handy on your phone — you'll create the bot there.
- NVIDIA drivers must already be installed (they are, since both cards show up). Confirm with the GPU check in Stage 1.

Stage 0 — Tailscale (admin reach)

You already get in via AnyDesk, so this is optional but recommended: it gives you a stable private address to reach the box (RDP/AnyDesk/SSH) from anywhere without port-forwarding. It joins your existing tailnet.

```
winget install --id tailscale.tailscale -e
tailscale up      # prints a login URL; open it and authenticate to your tailnet
tailscale ip -4   # note the 100.x.y.z address for this PC
```

Note: Tailscale's built-in SSH server doesn't run on Windows. For admin from the Mac, use **AnyDesk** or **Windows RDP over the Tailscale IP** — both work fine.

Stage 1 — Ollama + Ornith on the RTX 4000

Install Ollama and confirm both GPUs are visible:

```
winget install --id Ollama.Ollama -e
ollama --version
nvidia-smi -L
```

nvidia-smi -L lists each GPU with an index, e.g. *GPU 0: Quadro RTX 4000* and *GPU 1: GeForce GTX 1060*.

Write down which index is the RTX 4000 — you need it next. (Indexes can be either order depending on slot.)

Pin the default Ollama to the RTX 4000 only, and keep the model resident so the first Telegram reply isn't slow.

Replace 0 below with the RTX 4000's index if it wasn't 0:

```
[Environment]::SetEnvironmentVariable("CUDA_DEVICE_ORDER", "PCI_BUS_ID", "User")
[Environment]::SetEnvironmentVariable("CUDA_VISIBLE_DEVICES", "0", "User")
[Environment]::SetEnvironmentVariable("OLLAMA_KEEP_ALIVE", "-1", "User")
```

Now fully quit Ollama (system tray → right-click → Quit) and relaunch it from the Start menu so it picks up those variables. Then pull and test Ornith:

```
ollama pull ornith:9b
ollama run ornith:9b "Reply with one short sentence to confirm you are working."
```

Confirm it loaded on the RTX 4000 and not the CPU:

```
ollama ps          # PROCESSOR column should read 100% GPU
nvidia-smi         # RTX 4000 should show a python/ollama process using ~6 GB
```

■ If ollama ps shows any CPU split, the model spilled out of VRAM. Lower the context or confirm CUDA_VISIBLE_DEVICES points only at the 8 GB card. Ornith 9B (5.6 GB) fits with room for a normal context window, but the 256K max context will not fit — keep context modest.

Stage 2 — Hermes, pointed at local Ollama

Install the Hermes CLI (this also installs its Python/Node dependencies):

```
iex (irm https://hermes-agent.nousresearch.com/install.ps1)
```

Close and reopen PowerShell so hermes is on your PATH. Point its main model at Ollama's OpenAI-compatible endpoint:

```
hermes config set model.provider custom
hermes config set model.base_url http://localhost:11434/v1
hermes config set model.default ornith:9b
```

Ollama ignores the API key but Hermes wants one present. Create %USERPROFILE%\hermes\env with a dummy key (Notepad is fine):

```
OPENAI_API_KEY=ollama
OPENAI_BASE_URL=http://localhost:11434/v1
```

For reference, the equivalent block in %USERPROFILE%\hermes\config.yaml looks like this:

```
model:
  provider: custom
  base_url: "http://localhost:11434/v1"
  default: "ornith:9b"
```

Launch Hermes in the terminal and chat with it to confirm Ornith answers:

```
hermes
```

Note: If Hermes errors on connection, check Ollama is up: open <http://localhost:11434> in a browser on the PC — it should say “Ollama is running.”

Stage 3 — Telegram gateway

On your phone, in Telegram:

- Open **@BotFather**, send `/newbot`, give it a name, and a username ending in `bot`.
- BotFather replies with a token like `123456789:ABCdef...` — copy it.
- Message **@userinfobot**; it replies with your numeric user ID (e.g. `123456789`).

Add both to `%USERPROFILE%\hermes\.env` (alongside the dummy key from Stage 2):

```
TELEGRAM_BOT_TOKEN=123456789:ABCdefGHIjklMNOpqrSTUvwxyz  
TELEGRAM_ALLOWED_USERS=123456789
```

■ Always set `TELEGRAM_ALLOWED_USERS` — without it the gateway denies everyone, and the bot token alone would otherwise let strangers talk to your brain. Comma-separate to add more people later.

Enable and start the gateway:

```
hermes gateway setup      # interactive; pick Telegram when prompted  
hermes gateway            # starts the bot; comes online in seconds
```

Text your bot from Telegram. A reply means the whole chain works: phone → Telegram → Hermes → Ornith on the RTX 4000. **This is your working brain — stop here until it's rock solid.**

Stage 4 — GTX 1060 sidecar (optional)

Run a second Ollama instance locked to the GTX 1060 for the lightweight auxiliary work (small model, embeddings) so it never competes with Ornith for the RTX 4000. Use the GTX 1060's index from `nvidia-smi -L` (shown as 1 below).

In a dedicated PowerShell window, start instance #2 on port 11435:

```
$env:CUDA_DEVICE_ORDER="PCI_BUS_ID"  
$env:CUDA_VISIBLE_DEVICES="1"          # GTX 1060 index  
$env:OLLAMA_HOST="127.0.0.1:11435"  
ollama serve
```

Leave that running. In a second window, pull models into instance #2:

```
$env:OLLAMA_HOST="127.0.0.1:11435"  
ollama pull llama3.2:3b  
ollama pull nomic-embed-text
```

Then route Hermes' auxiliary tasks at it in `config.yaml`. The verified pattern is the vision block below; mirror it for other auxiliary tasks. Confirm the exact auxiliary subkeys your Hermes build exposes with hermes config before relying on embeddings routing:

```
auxiliary:
  vision:
    provider: custom
    base_url: "http://localhost:11435/v1"
    model: "llama3.2:3b"
  # embeddings: (mirror the same three keys, model: nomic-embed-text)
```

Note: Whisper / voice: Ollama does not serve Whisper, so the 1060 won't do speech-to-text through Ollama. Hermes handles Telegram voice on its own; if you later want local transcription you'd add a separate faster-whisper service on the 1060. Treat that as a future add-on, not part of this build.

Stage 5 — Make it survive reboots

The default Ollama (instance #1) already auto-starts at login. The 1060 instance and the Hermes gateway do not — register them as startup tasks. Because GPU/CUDA work is happiest in a real user session, the reliable path on a headless box is **auto-login + “at log on” tasks**.

Create a startup script, e.g. C:\hermes\start-brain.ps1:

```
# start-brain.ps1
# (only needed if you set up the Stage 4 sidecar)
$env:CUDA_DEVICE_ORDER="PCI_BUS_ID"
$env:CUDA_VISIBLE_DEVICES="1"
$env:OLLAMA_HOST="127.0.0.1:11435"
Start-Process ollama -ArgumentList "serve" -WindowStyle Hidden
Start-Sleep -Seconds 5
Start-Process hermes -ArgumentList "gateway" -WindowStyle Hidden
```

Register it to run at log on (run once, as admin):

```
schtasks /Create /TN "HermesBrain" /SC ONLOGON /RL HIGHEST ^
/TR "powershell -ExecutionPolicy Bypass -File C:\hermes\start-brain.ps1"
```

Set the PC to auto-login (so the session exists after a power blip), then reboot once and confirm the bot comes back online by itself.

Note: Prefer a true Windows service instead of auto-login? Use **NSSM** (winget install NSSM.NSSM) to wrap ollama serve and hermes gateway as services — but verify CUDA still engages under the service account (Session 0 sometimes can't see the GPU). Auto-login is the lower-risk default.

Appendix — quick checks & gotchas

Verify what's loaded and where:

```
ollama ps # instance #1 (11434)
$env:OLLAMA_HOST="127.0.0.1:11435"; ollama ps # instance #2 (1060)
nvidia-smi # per-GPU memory + processes
```

Common snags:

- **Hermes can't reach the model** → confirm <http://localhost:11434> says “Ollama is running,” and that `base_url` ends in `/v1`.
- **Model on CPU / slow** → `ollama ps` not 100% GPU; check `CUDA_VISIBLE_DEVICES` and that context isn't cranked toward 256K.

- **Bot silent** → wrong/empty TELEGRAM_ALLOWED_USERS, or token typo; restart hermes gateway after editing .env.
- **Both GPUs fighting over one model** → you forgot to pin; each Ollama instance must have its own CUDA_VISIBLE_DEVICES and OLLAMA_HOST.
- **Quality feels weak on hard tasks** → expected for a local 9B; add an OpenRouter/Portal key as a second provider for heavy jobs and keep Ornith for everyday/private.

Commands verified against the Hermes Agent docs (install, model config, Telegram gateway) on build date. Ollama/CUDA flags are standard. — Nyquist